

LambdaByte

Eight-bit arithmetic in lambda calculus

Binary arithmetic and overflow detection in lambda calculus

1. Purpose and syntax

The project preserves the proposed binary representation and makes it usable in a small language. Ordinary byte arithmetic becomes a network of lambda-encoded Boolean gates, adders, and a widened multiplier. Scala implements the general lambda evaluator and the external input/output; the arithmetic and its flags are lambda terms, not native arithmetic instructions.

The two layers are important: the **language core** is unrestricted untyped lambda calculus; the **numeric library** supplies eight-bit words with overflow detection.

```
let x = 200;
print x + 100;
let increment = fun n -> n + 1;
print increment(41);
print if is_zero(0) then 7 else 9;
```

Compact grammar ({...} means repetition; [...] means optional):

```
program ::= { ';' | statement ';' } [statement]
statement ::= 'let' definition | ['print'] expr
definition ::= ['rec'] id { id } '=' expr
expr ::= 'fun' id { id } '->' expr
      | 'if' expr 'then' expr 'else' expr
      | 'let' definition 'in' expr | sum
sum ::= product { ('+' | '-') product }
product ::= apply { '*' apply }
apply ::= atom { '(' expr ')' }
atom ::= number | id | '(' expr ')' | '-' apply
```

In the implementation, semicolons separate statements; only the final semicolon is optional. * binds more tightly than +/-, which associate left. Identifiers begin with a letter or underscore. Line comments begin with # or //. Keywords cannot be used as ordinary binding names. Negative literals have a special atom rule so they are encoded directly rather than treated as subtraction.

Function bindings may use let f x = body, which expands to a lambda abstraction. Local let x = e in body expands to (lambda x.body) e. let rec expands the definition using FIX, so recursion remains part of the lambda core.

Decimal literals range from 0 to 255; binary literals use 0b. Negative literals range from -128 to -1 and elaborate to their two's-complement bytes. Every numeric output displays both interpretations: unsigned 0..255 and signed -128..127.

2. Binary representation and foundational definitions

Let bits b_0 through b_7 describe a byte, with b_0 least significant. Its term accepts handlers f_7 through f_0 and a seed x . Each set bit applies its handler once; each clear bit skips it. Applications are ordered from high bits outside to low bits inside. In the user's names, $a=f_7$, $b=f_6$, ..., $h=f_0$.

```
Zero  := λa b c d e f g h x. x
One   := λa b c d e f g h x. h x
Two   := λa b c d e f g h x. g x
Three := λa b c d e f g h x. g (h x)
```

This is a binary selection encoding, not a unary Church numeral. The canonical representation has nine abstractions and at most eight selected applications.

```
TRUE  := λt f. t          FALSE := λt f. f
I      := λx. x           MARK  := λx. TRUE
NOT    := λp. p FALSE TRUE
AND    := λp q. p q FALSE
OR     := λp q. p TRUE  q
XOR    := λp q. p (NOT q)
```

TRUE and FALSE choose a branch. I preserves the seed. MARK makes the seed TRUE when a selected handler is visited. The remaining definitions are the standard Boolean operations expressed as branch selection.

Bit extraction. $B8_i$ passes MARK in position i , I in the other seven positions, and FALSE as the seed. If the bit is absent, the result stays FALSE. If present, MARK produces TRUE. For example:

```
B8_0 := λn. n I I I I I I I MARK FALSE
B8_1 := λn. n I I I I I I MARK I FALSE
B8_7 := λn. n MARK I I I I I I I FALSE
```

These probes also show that distinct canonical bytes are distinguishable: at least one bit probe returns different Booleans.

Packing. PACK8 takes eight Boolean bits and returns the original representation:

```
PACK8 := λb0 b1 b2 b3 b4 b5 b6 b7.
  λf7 f6 f5 f4 f3 f2 f1 f0 x.
  (b7 f7 I) ((b6 f6 I) ((b5 f5 I) ((b4 f4 I)
    ((b3 f3 I) ((b2 f2 I) ((b1 f1 I) ((b0 f0 I) x)))))))
```

The same definitions extend mechanically to sixteen positions for intermediates. All concrete terms, including every probe and packer, are exported in `core.lambda`.

3. Results, addition, and subtraction

A result contains a word and two encoded Boolean flags:

```
RESULT := λword u s k. k word u s
VALUE  := λr. r (λword u s. word)
UF     := λr. r (λword u s. u)
SF     := λr. r (λword u s. s)
```

UF means unsigned carry, borrow, or product truncation, depending on the operation. SF means signed overflow. An input literal has both flags FALSE.

Full adder. Given operand bits p , q and incoming carry c :

```
t      = XOR p q
sum    = XOR t c
carry  = OR (AND p q) (AND t c)

FA := λp q c k.
      (λt. k (XOR t c) (OR (AND p q) (AND t c))) (XOR p q)
```

The continuation k receives the sum bit and carry. For all eight Boolean input triples, $p+q+c = \text{sum} + 2*\text{carry}$. Chaining this identity over eight positions proves that the ripple adder's low byte is correct and $c8$ records the ninth bit.

ADC8 m n $c0$ chains FA from bit 0 to bit 7. Each continuation binds si and $c(i+1)$. The final continuation returns:

```
RESULT (PACK8 s0 s1 s2 s3 s4 s5 s6 s7) c8 (XOR c7 c8)
ADD_RAW := λm n. ADC8 m n FALSE
```

Unsigned addition overflows exactly when $c8$ is TRUE. Signed addition overflows exactly when the carry into the sign position differs from the carry out, hence $\text{XOR } c7 \ c8$. These are distinct properties.

Subtraction. INV8 flips each bit and repacks it. Since an eight-bit complement represents $255-n$, the circuit calculates $m + \text{INV8}(n) + 1$, with initial carry TRUE:

```
SUB_WORD := λm n. VALUE (ADC8 m (INV8 n) TRUE)
SUB_RAW  := λm n.
      (λr. RESULT (VALUE r) (NOT (UF r)) (SF r))
      (ADC8 m (INV8 n) TRUE)
```

The final carry means no unsigned borrow, so SUB_RAW negates it. The carry-XOR test also detects signed subtraction overflow. SUB_WORD returns only the wrapped word, for internal modular calculations where those flags are intentionally ignored.

4. Multiplication and overflow

Shifting a word left replaces the highest handler with I, moves the remaining handlers up one position, and ignores the new low handler:

```
SHL8 := λn f7 f6 f5 f4 f3 f2 f1 f0 x.
      n I f7 f6 f5 f4 f3 f2 f1 x
```

SHL16 does the same over sixteen positions. WIDEN8 puts the original eight bits in the low half of a sixteen-bit word and FALSE in the high half. ADC16 is a sixteen-position ripple adder that returns only its low sixteen bits. ADD16 starts it with FALSE carry.

The numeral itself selects partial sums:

```
MUL16 := λm n.
  m (ADD16 (SHL16^15 n)) ... (ADD16 (SHL16 n))
  (ADD16 n) ZERO16
```

The ellipsis and superscripts here are finite notation only: core.lambda contains all sixteen arguments and every repeated shift explicitly. A selected bit i adds $2^i \cdot n$ to the accumulator. For widened byte inputs, the exact unsigned product is at most $255 \cdot 255 = 65025$, so it fits in sixteen bits without loss.

LOW8 and HIGH8 extract and repack the two halves. Let p be the full unsigned product, lo its low byte and hi its high byte. NONZERO8 hi is the unsigned fault flag: it is TRUE exactly when p exceeds 255.

Signed overflow needs another test. An unsigned high byte alone is insufficient: the raw byte 255 represents signed -1, so $255 \cdot 1$ must not report signed overflow. Let sm and sn be the sign bits of the operands. Then:

```
m_signed = m_unsigned - 256*sm
n_signed = n_unsigned - 256*sn
signed_product mod 65536
= unsigned_product - 256*(sm*n_unsigned + sn*m_unsigned)
```

The omitted $65536 \cdot sm \cdot sn$ term is zero modulo 65536. Thus the high byte of the signed product is reconstructed using existing lambda operations:

```
hs = SUB_WORD (SUB_WORD hi (sm n ZERO8)) (sn m ZERO8)
```

SIGNED_FIT lo hs tests whether every bit of hs equals bit 7 of lo . That is exactly the sign-extension condition for a sixteen-bit signed value to fit in eight bits. MUL_RAW returns RESULT lo (NONZERO8 hi) (NOT (SIGNED_FIT lo hs)).

For canonical byte inputs, all low results and both flags are exact. The high-byte correction is modular arithmetic, so its own discarded flags must not contaminate the externally reported multiplication flags.

5. Language semantics and evaluator

Source expressions elaborate into just three AST constructors: Var, Lam, App. Abstraction and application translate directly. A conditional becomes `p yes no`. A literal becomes `RESULT encoded_word FALSE FALSE`.

`ADD_LIFT`, `SUB_LIFT`, and `MUL_LIFT` first project both operand words, call the matching RAW operation, and combine each new flag with both operand flags using OR:

```
value_out = VALUE raw
u_out = OR (UF a) (OR (UF b) (UF raw))
s_out = OR (SF a) (OR (SF b) (SF raw))
```

The output is a `RESULT` term. A later safe arithmetic operation does not clear an earlier flag. Bindings preserve the whole result, not merely its byte. Predicates return plain Church Booleans: `ZERO_Q` tests whether the byte is zero; `EQ_Q` compares bytes; `HAS_FAULT` combines UF and SF. Predicate history is not attached to a branch.

FIX is Curry's fixed-point combinator:

```
FIX := λf. (λx. f (x x)) (λx. f (x x))
```

The evaluator is call-by-need: an application evaluates its function and saves its argument in a thunk. A closure binds that thunk in its captured lexical environment. The first demand evaluates and caches the argument. Discarded arguments and unchosen branches stay unevaluated. This supports FIX without a CBV adaptation.

Normal-form readback applies closures to fresh neutral variables and reconstructs the lambda AST. Fresh names cannot collide with source identifiers. Output decoding then recognizes a canonical `RESULT`, its word, and its flags. Arbitrary normalized lambda terms remain printable; they are not forced into the numeric representation.

Expressiveness. Any untyped lambda term embeds directly: `x` becomes `x`, `lambda x.t` becomes `fun x -> E(t)`, and `t u` becomes `(E(t))(E(u))`. Conversely, expanding the finite numeric helpers yields ordinary lambda terms. Apart from external I/O and practical resource limits, the expression layer has the expressive power of untyped lambda calculus. The fixed-size numeric library is not, by itself, the reason for that expressiveness.

Limits. Numeric outputs wrap modulo 256; flags do not trap or preserve a larger answer. The language has no division, arbitrary-precision numbers, strings, mutable state, or language-level file operations. It is untyped: arithmetic expects canonical numeric results, and misuse can return an unexpected term or fail to terminate. Fuel, stack, and memory constrain execution. Full normalization can demand function bodies. Ordinary `let` is nonrecursive; use `let rec` or `FIX`. Rebinding does not alter earlier closures.

6. Definition catalog, execution, and verification

Each generated family below denotes concrete, finite lambda ASTs, not a primitive operation in the evaluator. The exact exported definitions are in `core.lambda`.

Definitions	Role
TRUE, FALSE, I, MARK	Boolean choice, identity, bit-presence probe
NOT, AND, OR, XOR	Boolean gates
RESULT, VALUE, UF, SF	Encoded numeric result and projections
ZERO8, ZERO16	Empty handler compositions
B8_0..B8_7, B16_0..B16_15	Extract the named position as a Church Boolean
PACK8, PACK16	Reconstruct handler compositions from Boolean bits
SHL8, SHL16	Shift left, dropping the highest bit
FA, ADC8, ADC16	One-position and ripple-carry adders
INV8, ADD_RAW, SUB_RAW, SUB_WORD	Complement, flagged arithmetic, wrapped subtraction
ADD16, MUL16	Finite sixteen-bit intermediate arithmetic
WIDEN8, LOW8, HIGH8	Zero extension and byte extraction
NONZERO8, EQ8, SIGNED_FIT	Bit tests for comparisons and product overflow
MUL_RAW	Low-byte multiplication with both exact flags
ADD_LIFT, SUB_LIFT, MUL_LIFT	Propagate operand flag history
ZERO_Q, EQ_Q, HAS_FAULT	Source-level numeric predicates
FIX	Unrestricted lazy recursion

Run the included executable with Java 17 or newer:

```
java -jar lambdabyte.jar
java -jar lambdabyte.jar examples/demo.lb
java -jar lambdabyte.jar --defs
java -cp lambdabyte.jar LambdaByteTests --exhaustive
```

The implementation was compiled with Scala 2.11.12 and tested on Java 17. No Scala installation is required for the packaged JAR. Source build instructions are in `README.md`. Scala's standard library is included with its attribution.

Exhaustive tests checked all 65536 operand pairs per operation against an independent integer oracle, verifying the byte and both flags. They also checked all 256 numeral encodings and 2048 bit probes. The arithmetic definitions match that verified core, with only the combined-flag helper renamed. This syntax revision passed 3209 checks, covering precedence, literals, flags, laziness, scope, recursion, local bindings, malformed input, and evaluation limits. Both records are in `validation.txt`.

References and attribution

The binary selection encoding is based on the design proposed in this conversation. The gates, adders, multiplier, and signed correction are derived and implemented here.

1. Cornell CS3110, Lecture 27: Introduction to the lambda calculus (2011). <https://www.cs.cornell.edu/courses/cs3110/2011sp/Lectures/lec27-lambda/lambda.htm> Background for lambda terms, Church Booleans, and encoded data.

2. Scala documentation: Getting Started. <https://docs.scala-lang.org/getting-started/install-scala.html> Compiler and runner installation guidance.